

Object-oriented programming (OOP):

Object-oriented programming (OOP) is a programming paradigm centered around the concept of **objects**, which can encapsulate both data and methods that operate on that data. This model organizes software design around these objects rather than focusing solely on functions and logic.

Major Feature of OOP are as following:

Class:

- A **class** in C++ is like a **template** or **blueprint** for creating objects. It defines the properties and behaviors that the objects created from this class will have.
- **Properties:** These are variables (or data members) that store information about the object. For example, in a Car class, properties might include wheels, speed, and mileage.
- **Behaviors:** These are functions (or member functions) that define what actions the object can perform. For example, the Car class might have behaviors like `accelerate()` to speed up the car or `applyBrakes()` to stop the car.

Example:

```
class Car {  
    int speed; // property  
    int mileage; // property  
    char colour; //property  
  
public:  
    void applyBrakes(); // behavior (function)  
    void setMileage(int m) {  
        int mileage = m; } // behavior (function)  
};
```

Object:

- An **object** is an **instance** of a class. It is a specific, actual item created from the class.
- Once you define a class, no memory is used, but when you create an object (instance) from it, memory is reserved.
- **Example:** If you have a Car class, creating a specific car object like `myCar` (with a speed of 100 km/h) is an instance of the class.

Example

```
Car myCar; // myCar is an object of the Car class
```

Encapsulation:

- **Encapsulation** is the concept of **bundling** or **grouping** data (properties) and methods (behaviors) together in one unit (the class).
- It also involves **restricting** access to some of an object's components to protect the object's integrity. This can be done by setting some data to be **private** so it can only be accessed by certain functions, not directly from outside the class.
- **Example:** Imagine a bank account where the balance is hidden (encapsulated). Only authorized functions (like deposit or withdraw) can change the balance. Others can't directly change it.

Example:

```
class BankAccount {  
    int balance; // hidden data  
public:  
    void deposit(int amount) { balance += amount; } // function to change data  
    int getBalance() { return balance; } // function to access data  
};
```

Here, you can't directly change the balance variable from outside the class. You can only use the functions deposit() or getBalance().

Abstraction:

- **Abstraction** is the concept of hiding the **complex details** and showing only the **essential** information to the user.
- It makes it easier to interact with the system without needing to know how it works behind the scenes.
- **Example:** When you drive a car, you press the **accelerator** to speed up. You don't need to understand how the accelerator works inside the engine to drive the car this is abstraction. In programming, this is like calling a function (pow()) without needing to know the exact code for how it calculates powers.

Polymorphism:

- **Polymorphism** means "many forms." It allows one action or method to behave in different ways depending on the situation.

There are two types of polymorphism:

1. **Compile Time Polymorphism:** Compile-time polymorphism is done by overloading an **operator or function**. It is also known as "static" or "early binding".
2. **Runtime polymorphism:** Runtime polymorphism occurs when functions are resolved at runtime rather than compile time when a call to an **overridden** method is resolved dynamically at runtime rather than compile time. It's also known as **late binding** or **dynamic binding**.

Inheritance:

- **Inheritance** allows a class to inherit properties and methods from another class. This helps **reuse** code and build on existing functionality.
- The class that **inherits** is called the **subclass** or **derived class**. The class being inherited from is called the **base class** or **superclass**.
- **Example:** Imagine a base class called Animal. You can create subclasses like Dog, Cat, or Cow, which inherit properties and behaviors from Animal. For example, Dog and Cat might share common behaviors like eat() or sleep(), but they can also have their own unique behaviors.

Example:

```
class Animal {  
    int height;  
    char name;  
  
    public:  
        void eat();  
        void sleep();  
};  
class Dog : public Animal {  
    public:  
        void bark(); };
```

Here, Dog inherits the all properties and behaviors from Animal and adds its own unique behaviors bark().

Message Passing:

- In OOP, objects communicate with each other by sending and receiving messages. A message is essentially a request for an object to execute one of its methods.
- When object A wants object B to perform an action, object A sends a message to object B. This message typically includes the name of the method to be invoked and any necessary arguments (data). Object B receives the message and executes the corresponding method.
- This concept underlies how object interactions are modeled. For example, a Customer object might send a deposit(amount) message to a BankAccount object.

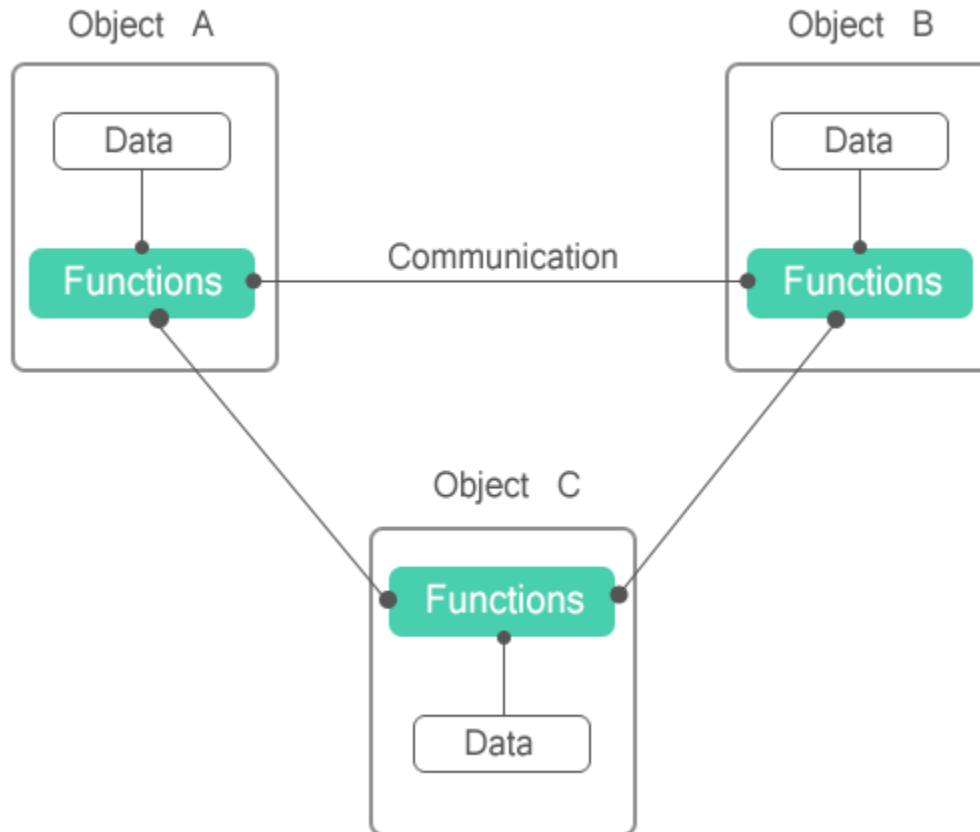
OOP vs POP

Aspect	Object-Oriented Programming (OOP)	Procedural-Oriented Programming (POP)
Paradigm	Based on objects (data and methods bundled together).	Based on procedures or routines (functions).
Key Focus	Focuses on objects that represent real-world entities.	Focuses on the sequence of tasks or actions.
Modularity	High modularity due to encapsulation.	Low modularity, as the code is mostly sequential.
Data Security	Data is encapsulated within objects.	Data is separate from functions.
Code Reusability	Encourages reusability through inheritance and polymorphism.	Limited reusability, relies on functions or procedures.
Approach	Top-down or bottom-up (using objects).	Top-down approach (focusing on the sequence of steps).
Inheritance	Supports inheritance, allowing the creation of new classes.	No direct support for inheritance.
Polymorphism	Supports polymorphism (one method can behave differently).	No polymorphism.
Abstraction	High level of abstraction; details are hidden within objects.	Limited abstraction; relies on function implementation.
Maintenance	Easier maintenance, as changes in one object can be isolated.	Maintenance can be difficult as changes may affect the entire flow.
Examples Languages	Java, C++, Python, C#, Ruby, Swift.	C, Fortran, Pascal, Basic.

Object-Oriented programming paradigm:

Object-Oriented programming (OOP) paradigm is a programming nature that organizes software design around "**objects**" instead of functions and logic. These objects represent real-world entities, concepts, or processes, and they bundle **data** (attributes) and **actions** (methods) into reusable, self-contained units. This approach makes code more modular, scalable, and easier to manage, especially for large or complex systems.

Organization of data and function in OOP is shown below:



Striking Features of Object-Oriented Programming (OOP):

1. **Emphasis on Data rather Than Procedures:** Unlike procedural programming where the focus is on functions, OOP emphasizes **data** and the operations that manipulate it.
2. **Programs Are Divided into Objects:** OOP organizes software as a collection of **objects**, each representing a distinct part of the program (real-world entities or concepts).
3. **Data Structures Represent Objects:** Objects are designed using **classes** that define their **attributes (data)** and **methods (functions)**—effectively characterizing their behavior and state.

4. **Data Hiding (Encapsulation):**

- Data is hidden within objects and protected from unauthorized access using **access specifiers** (like private, public, and protected).
- External code cannot directly access internal object data—only through defined methods.

5. **Objects Communicate via Functions:** Objects interact with one another by **calling methods** (message passing), promoting modularity and separation of concerns.

6. **Easy Addition of New Data and Functions:** OOP supports **extensibility** new attributes and behaviors can be added easily without modifying existing code, thanks to features like **inheritance** and **polymorphism**.

7. **Follows Bottom-Up Approach:** Program design typically begins with building **small, reusable objects**, which are later combined to form complex systems—opposite of the **top-down** approach in procedural programming.

A Way of Viewing the World: *Understanding Object-Oriented Programming through a Real-World Example*

Object-Oriented Programming (OOP) becomes easier to understand when related to real-life situations. Let's use the example of a food delivery service to explore the core OOP concepts.

Real-World Scenario:

I am at my office and I want to send food to my family members who are at home. The food should come from a particular hotel. Since I can't leave the office due to distance and time constraints, I need a way to solve this.

To address the issue, I contact Foodmandu, an online food delivery service. I tell them what food I want, how much, and from which hotel it should be delivered. Foodmandu handles the rest and delivers the food to my family.

Now let's relate this to object-oriented programming:

Agents and Communities

In OOP, a program is structured as a community of interacting agents, known as objects. Each object is responsible for certain tasks and provides services to others.

- In our example, the online food delivery ecosystem is the **community**.
- Foodmandu and different hotels are the **agents** (objects).

Each hotel provides food-related services, while Foodmandu handles requests and delivery logistics.

Messages and Methods

Objects in OOP communicate by sending messages. A message triggers a method in the receiving object.

I (the user object) send a message to Foodmandu, which includes:

- The type of food
- The quantity
- The hotel name

Foodmandu receives the message and invokes methods to:

- Contact the hotel
- Place the order
- Assign a delivery person
- Deliver the food to my family

Responsibilities

In OOP, each object has responsibilities: the actions or behaviors it is expected to perform.

- In our example, my request simply defines the goal: food delivered to my family.
- Foodmandu, as the responsible agent, decides how to achieve this internally managing logistics and execution.
- This way, I don't need to know how Foodmandu does it; I just need to send the request. This is the essence of abstraction.

Classes and Instances

In OOP, all objects are instances of classes. A class defines a blueprint (data and behavior) for its instances.

- Foodmandu can be viewed as a class with behaviors like: ***placeOrder()***, ***assignDelivery()***, and ***trackOrder()***.
- Each order I place is an instance of that class.
- Hotels can be instances of a Hotel class, each with its own menu and preparation time.

This simple food delivery scenario reflects the core concepts of object-oriented programming:

- Objects interact through messages
- Each object has specific responsibilities
- Objects are instances of classes that define their behavior

Object-Oriented Analysis and Design (OOAD):

OOAD is based on the concepts of object-oriented programming (OOP) and is an organized and systematic approach to designing and developing software systems. It is a software engineering paradigm that integrates two distinct but closely related processes: Object-Oriented Analysis (OOA) and Object-Oriented Design (OOD).

Object-Oriented Analysis:

Object-Oriented Analysis (OOA) is the process of understanding and analyzing the system requirements by looking at the problem scenario in terms of objects.

- These objects represent real-world entities or concepts that are relevant to the system being developed.
- During OOA, the goal is to identify the objects, their attributes, behaviors, and relationships, without focusing on how the system will be implemented.

For example: Let's say you're building a game:

- OOA helps you figure out all the things you need to know about the game world the characters, their features, and how they interact.
- It's like making a map of everything important.
- OOA also helps you understand what your game characters will do. It's like writing down a script for each character.

Object-Oriented Design:

In the object-oriented software development process, the analysis model, which is initially formed through object-oriented analysis (OOA), undergoes a transformation during object-oriented design (OOD) (***i.e implementation of the conceptual model developed in OOA***). This evolution is crucial because it shapes the analysis model into a detailed design model.

Furthermore, as part of the object-oriented design process, it is essential to define specific aspects:

- **Data Organization of Attributes:** OOD involves specifying how data attributes are organized within the objects. This includes determining the types of data each object will hold and how they relate to one another.
- **Procedural Description of Operations:** OOD requires a procedural description for each operation that an object can perform. This involves detailing the steps or processes involved in carrying out specific tasks.

Responsibility-Driven Design (RDD):

Responsibility-Driven Design (RDD) is an object-oriented design methodology that emphasizes the responsibilities of objects in a system, rather than focusing primarily on data structures or processes.

It was popularized by Rebecca Wirfs-Brock in the 1990s and remains influential in thinking about object design, especially in agile and clean code practices.

In RDD, the main idea is that objects are responsible for knowing something and doing something. Instead of thinking "*What data does this object hold?*" or "*What operations do we need?*" RDD asks:

- *What is this object's responsibility?*
- *Who does it collaborate with to fulfill that responsibility?*

The focus shifts to the behavior and collaboration of objects, which makes the design more maintainable, reusable, and easier to change.

Core Concepts of RDD

1. Responsibilities

These are the things an object is expected to know or expected to do. They're often divided into:

- a) **Knowing:** Information the object maintains (e.g., a Customer knows their name and address).
- b) **Doing:** Actions the object performs (e.g., a Cart object calculates totals or applies discounts).

2. Collaborators

Objects rarely do everything themselves—they delegate. Collaborators are the other objects an object interacts with to fulfill its responsibilities.

3. Roles

An object may take on different roles depending on the context. A single class might serve multiple responsibilities when reused in different parts of the system.

Techniques Used in RDD:

1. CRC Cards (Class-Responsibility-Collaborator):

One of the key tools in RDD, CRC cards help teams brainstorm and visualize object responsibilities and their collaborators. Each card contains:

- a) Class name
- b) Responsibilities (what it does or knows)
- c) Collaborators (other objects it works with)

This helps encourage discussion about design in a lightweight, interactive way.

2. Use-Case and Scenario Analysis

RDD often begins with use cases or scenarios, helping to define how the system should behave from a user perspective. This helps identify the responsibilities needed to fulfill those scenarios.

CRC Cards:

CRC (Class, Responsibility and Collaboration) cards are a design tool used in object-oriented programming to capture and organize information about classes, their responsibilities, and their collaborations within a software system.

Components of CRC Cards

CRC (Class-Responsibility-Collaboration) cards consist of three main components:

- **Class:** The class represents a software entity or component in the system. It encapsulates data (attributes or properties) and behavior (methods or functions) related to a specific concept or functionality. On a CRC card, the class name is typically written at the top.
- **Responsibility:** Responsibility refers to the tasks or functions that a class is responsible for performing. It defines what the class does or what it is expected to accomplish within the system. Responsibilities are listed on the card below the class name, often as bullet points or short phrases.
- **Collaboration:** Collaboration describes the interactions or relationships that a class has with other classes in the system. It specifies how the class communicates or cooperates with other classes to fulfill its responsibilities. Collaborations are listed on the card alongside the responsibilities, indicating the classes with which the current class interacts.



These components work together to provide a concise and structured representation of the class's role, behavior, and relationships within the system. By organizing this information on index cards, developers can visually analyze and discuss the design of the system, identify potential design flaws, and iteratively refine the system's architecture.

The purpose of CRC cards in software design include:

- **Clarify Class Responsibilities:** CRC cards help identify and define the responsibilities of each class in the system.

By listing the responsibilities of a class on a card, developers can gain a clear understanding of what each class is supposed to do.

- **Identify Collaborations:** CRC cards also specify the collaborations between classes. This includes identifying which classes need to interact with each other to fulfill their responsibilities.

By outlining these collaborations, developers can ensure that the interactions between classes are well-defined and appropriate.

- **Encourage Communication and Collaboration:** CRC cards are typically used in group settings, such as brainstorming sessions or design meetings.

They encourage team members to actively participate in discussions about the system's design, fostering collaboration and communication among team members.

- **Iterative Design and Refinement:** CRC cards support an iterative approach to software design. As the design evolves and requirements change, CRC cards can be easily updated or revised to reflect these changes.

This allows for continuous refinement of the system's design throughout the development process.

Example of CRC cards: *Let's understand CRC Cards using example of **banking** system.*

Account	
• Store account holder information (name, address, contact details). • Maintain account balance. • Perform transactions (deposit, withdrawal). • Generate account statements.	Transaction Statement

Transaction	
• Record transaction details (date, type and amount). • Update account balance based on deposit or withdrawal operations.	Account Statement

Statement	
• Generate account statements for a specified period. • Include transaction details (date, type and amount) in the statement.	Account

Example of CRC cards: *Let's understand CRC Cards using example of **Class Student**.*

Student	
• Student's information (name, address, contact details). • Enroll in a semester • Request Transcript	Semester Transcript

Semester	
• Semesters' information (name, Number, Fees, Credit Hours). • Add Students • Drop Students • Assign Professor	Professor Student

Professor	
• Professor's information (name, address, contact details, Salary). • Conduct Lectures • Provides Study Materials • Grade Papers	Semester Student

Transcript	
• Student's information (name, address, contact details). • Marks Obtained • Final Grade	Semester Student

Responsibility Implies Non-Interference

In Object-Oriented Analysis and Design (OOAD), the concept that "***responsibility implies non-interference***" means that once an object is assigned a responsibility, it should have the autonomy to fulfill that responsibility without external interference.

This is similar to delegating a task to a person: after assigning the task, you trust them to complete it independently, rather than micromanaging every step.

For example:

In Real World, if you assign a child the task of cleaning their room, you do not stand over them and supervise each action. Instead, you expect the child to take responsibility and produce the desired outcome on their own.

In software, if an object is responsible for a specific action (like processing payments), other objects should not interfere with how it accomplishes that task they should simply trust it to deliver the result.

"Whenever responsibility is assigned to an object (programmer), no interference should be done afterwards. Responsibility implies a degree of independence or noninterference."

This principle leads to better encapsulation, modularity, and maintainability in OO systems.

Example: - Suppose you have a **Printer** object responsible for printing documents. Once a **Document Manager** object sends a print request, it should not interfere with how the Printer handles the printing process. The Printer manages its own internal steps (spooling, formatting, etc.) independently.

Programming in Small and Programming in Large:

In the context of Object-Oriented Analysis and Design (OOAD), the terms Programming in Small and Programming in Large refer to different approaches or scales of software design and implementation. These terms are often used to differentiate between developing small, modular systems versus large, complex systems.

Programming in Small:

Programming in Small refers to designing and implementing software systems that are relatively small in scope. In this context, the focus is on creating a limited set of functionalities with a clear, narrow scope. It typically involves less complex interactions between objects and fewer components. The design is more localized and manageable.

Key Characteristics:

- **Modularity:** Small-scale systems tend to focus on modular design where individual classes or components are designed to handle specific responsibilities.
- **Simplicity:** The number of objects, methods, and interactions is limited.
- **Focus on details:** Since the system is small, more attention is given to the fine details of the design of individual components.
- **Less complexity in relationships:** Inter-object relationships are often straightforward and direct.

Example: Imagine you're designing a Simple Library Management System. In this system, you might have the following classes:

1. **Book class** to represent a book with attributes like title, author, isbn.
2. **Member class** representing a library member with attributes like name, membershipID.
3. **Library class** to manage the list of books and members, and to handle basic operations like adding/removing books or members.

In this case, the system has a limited number of objects and interactions, making it easier to manage and understand.

Programming in Large:

Programming in Large refers to the development of larger, more complex systems where multiple objects, modules, and components need to work together. These systems often have higher degrees of abstraction, more intricate interactions, and need to consider scalability, flexibility, and maintainability.

Key Characteristics:

- **Complexity:** Systems tend to have many interacting components, classes, and modules.
- **Higher-level design:** There is a need for patterns, frameworks, and strategies to manage complexity.
- **Abstraction and Interfaces:** A higher degree of abstraction is often used to handle complexity, where different subsystems interact via well-defined interfaces.
- **Scalability:** The system is designed to grow over time, potentially adding new functionalities without disrupting the existing system.
- **Distributed Development:** Larger teams might be involved, requiring better documentation, communication, and division of work.

Example: Consider a Corporate Inventory Management System for a global business. This might include the following components:

1. **Product class** to represent different types of inventory items.
2. **Warehouse class** to represent various warehouses, each with its inventory.
3. **Order class** to manage orders made by customers.
4. **Shipping class** to track shipping details.
5. **Accounting system** to handle invoices and payments.
6. **User Roles** with different levels of access (e.g., admin, warehouse staff, customer service).

In this case, you need to deal with complex interdependencies, asynchronous operations, error handling, and maybe even distributed systems across different regions or time zones. This requires careful planning, design patterns like MVC (Model-View-Controller), SOLID principles, and so on.

Computation as Simulation:

In object-oriented programming (OOP), viewing computation as simulation emerges naturally from the paradigm's core philosophy. The fundamental idea is that software objects simulate or model real-world entities, behaviors, and interactions.

OOP was designed specifically to bridge the gap between how we conceptualize the world and how we represent it in code. When we create classes and objects, we're essentially creating digital models that simulate aspects of reality or abstract concepts.

Key Aspects of this View:

1. **Objects as Models:** Objects encapsulate both state (data) and behavior (methods), just as real-world entities have properties and capabilities. A Car object might have speed and fuel properties along with methods like `accelerate()` and `brake()`.
2. **Interactions through Messages:** In OOP, objects interact by sending messages to each other (method calls), simulating how entities communicate and affect each other in the world we're modeling.
3. **Inheritance as Classification:** Inheritance hierarchies mimic taxonomic classifications in the real world, where more specific entities inherit traits from more general categories.
4. **Polymorphism as Context-Sensitive Behavior:** Different objects responding to the same message in their own way simulates how different real-world entities might respond differently to the same stimulus.

Historical Context

This view has deep roots in OOP's origins. Simula, the first OO language, was explicitly designed for simulations. Alan Kay, who developed Smalltalk and coined the term "object-oriented," viewed objects as "little computers" that communicated with each other - essentially simulating distributed computation.

When we build complex OOP systems, we're creating intricate simulations where objects represent entities in our problem domain, and their interactions model the relationships and behaviors we're trying to capture in software.

Coping with complexity:

Complexity is one of the greatest challenges in modern software development. As systems grow, they tend to become increasingly difficult to understand, maintain, and extend.

Complexity makes systems harder to understand, maintain, and extend so Coping with complexity is critical for long-term success. "As systems evolve, complexity isn't just a side effect, it becomes a major engineering challenge."

The Nature of Complexity in Software:

Complexity in software systems stems from multiple sources. At its core, it reflects the inherent difficulty of mapping real-world problems into computational solutions. As Fred Brooks noted in his famous paper "No Silver Bullet," there are two fundamental types of complexity:

1. **Essential complexity:** refers to the inherent complexity of the problem domain itself. Some problems are simply complicated by nature like air traffic control systems or banking software that must handle countless regulations and edge cases. This complexity cannot be eliminated, only managed.
2. **Accidental complexity:** arises from our implementation choices and tools rather than from the problem itself. This might include overly complicated architectures, poor design decisions, or technologies that don't match the problem well. Unlike essential complexity, accidental complexity can and should be minimized.

Abstraction as a Response to Complexity

Abstraction is our primary tool for managing complexity. It works by creating simplified models that capture the essential aspects of a system while hiding unnecessary details.

Key Abstraction Mechanisms

1. Procedural Abstraction: The function or procedure represents one of our earliest abstraction mechanisms. When we define a function named `calculateTax()`,

we can use it without thinking about its internal workings. This creates an abstraction barrier a clear separation between what a component does (its interface) and how it accomplishes it (its implementation).

Procedural abstraction follows the principle that complexity is manageable when properly compartmentalized. Each procedure encapsulates a specific algorithm or operation, providing a single point of definition that can be reused throughout a system.

2. Data Abstraction: Data abstraction focuses on representing information without exposing its internal structure. Abstract data types (ADTs) like stacks, queues, and maps let us work with collections of data through well-defined operations without worrying about how the data is organized in memory.

For example, when using a dictionary or map data structure, we simply think about key-value associations rather than hashing algorithms or collision resolution strategies happening behind the scenes.

3. Object-Oriented Abstraction: Object-oriented programming combines data and procedural abstraction into coherent units that model real-world entities. This approach offers several powerful abstraction mechanisms:

- A. **Encapsulation** bundles data with the methods that operate on it, creating a self-contained unit with a well-defined interface. By hiding implementation details, encapsulation allows objects to evolve internally without affecting code that uses them.
- B. **Inheritance** establishes "is-a" relationships between classes, creating abstraction hierarchies where specialized classes inherit attributes and behaviors from more general ones. This enables us to describe common characteristics once and refine them as needed.
- C. **Polymorphism** allows different objects to respond to the same message in ways appropriate to their specific types. This enables us to write code that works with abstractions rather than specific implementations, providing tremendous flexibility.

4. Architectural Abstraction: At a higher level, architectural patterns provide abstraction mechanisms for organizing entire systems:

- A. **Layered architectures** separate systems into distinct levels of abstraction, where each layer provides a platform for the layer above it. This approach allows developers to focus on one level at a time.

- B. **Component-based architectures** organize systems as collections of pluggable components with well-defined interfaces, enabling complex systems to be built from simpler, reusable parts.
- C. **Service-oriented architectures** abstract functionality into independent services that communicate through standardized protocols, allowing systems to evolve and scale more easily.

Assignments:

- What is Software components? Explain different aspects of software components.
- What are the shortcoming of procedural Programming? Explain the notation of “Everything is an object” in an object oriented programming.
- Why Object-oriented Programming is Superior than Procedural-Oriented Programming. Explain.
- How object-oriented Programs are designed and developed according to the concept of RDD? Describe entire process in brief.
- Path follower Robot senses the path it needs to follow through its sensors. Based on the data received through its sensors, the robot make use of its actuators (Robotic Wheels) to steer itself forward.

For the above-mentioned systems, identify as many components (Collaborating objects) as you can draw CRC card for least three of them. Show the interaction between these components through interaction diagram.

- Write a short notes on:
 1. Interface and Implementation.
 2. Cohesion and coupling
 3. Non-linear behavior of Complexity